

Поддержка нечеткой логики для разработки требований

Ахмет Эгесой¹, и Айлин Гюэль²

¹ Доцент кафедры компьютерной инженерии Эгейского университета, Измир, Турция

² Аспирант кафедры компьютерной инженерии Эгейского университета, Измир, Турция

Корреспондент следует направлять Ахмету Эгесой ahmet.egesoy@ege.edu.tr

Авторские права © 2021 г. Сделано Ахмет Эгесой и др. Данная статья находится в открытом доступе и распространяется по лицензии Creative Commons Attribution, которая разрешает неограниченное использование, распространение и воспроизведение на любом носителе при условии надлежащего цитирования оригинальной работы.

АННОТАЦИЯ. Поскольку программные проекты становятся все более сложными, наибольшие риски в программной инженерии, скорее всего, происходят из ограничений неопытного разработчика, пытающегося представить себе границы абстрактного артефакта, который еще даже не существует. Инженерия требований чрезвычайно важна в проекте разработки программного обеспечения, но по своей сути сложна. Требования могут быть избыточными, необязательными, перекрывающимися и даже противоречивыми. Они поступают из разных источников и часто представлены неформально. Требования также складываются на протяжении всего процесса разработки и могут быть частично выполнены в разной степени на разных этапах процесса. В этой работе мы предлагаем нечеткую логику и модель для отслеживания требований и их выполнения. Мы также представляем логическую структуру менеджера базы знаний требований, которую мы создаем для упрощения инструментов быстрой разработки с учетом требований.

КЛЮЧЕВЫЕ СЛОВА — искусственный интеллект, нечеткая логика, инженерия требований, программная инженерия.

I. ВВЕДЕНИЕ

Инженерия требований, пожалуй, самая важная задача в проекте разработки программного обеспечения. Требования поступают от множества заинтересованных сторон с разными интересами, намерениями и мировоззрением. Это делает спецификацию требований сложной задачей не только потому, что она требует эффективных навыков коммуникации, но и потому, что требования могут быть взаимосвязаны неожиданным образом.

В базовой традиции разработки программного обеспечения, которая позже получила название каскадной модели процесса, проектирование требований естественным образом расматривается как первый этап процесса разработки. Более поздние методы разработки, использующие различные формы повторяющихся улучшений, предполагают, что проектирование требований продолжается на протяжении всего жизненного цикла системы. По мере развития системы к завершению требования постепенно выполняются. Некоторые из них могут отставаться в полуготовом состоянии в течение длительного времени.

Структурированные методы, работающие с функциональными требованиями, часто выстраивают их в иерархию. Этот подход также позволяет отделить необязательные требования от обязательных и сформировать мета-модель приемлемых конфигураций системы. Требования системы либо выполняются, либо нет.

Однако требования часто сложно отразить в жесткой структуре. Вместо этого они часто могут оказаться избыточными.

неполными, нечеткими, перекрывающимися и даже противоречивыми.

Во втором разделе обобщается существующая литература по применению нечеткой логики в программной инженерии. В третьем разделе содержатся некоторые теоретические замечания о нечеткой логике, а также о том, где и зачем она нужна.

В четвертом разделе представлено описание нашего проекта «Менеджер базы знаний требований», без вдавания в технические подробности, за исключением тех случаев, когда это необходимо для того, чтобы подчеркнуть некоторые интересные моменты, касающиеся нашего первоначального подхода к нечеткой логике. В пятом разделе содержатся выводы.

II. ЛИТЕРАТУРА НЕЧЕТКОСТИ В ТРЕБОВАНИЯХ К ИНЖЕНЕРИИ

Современная литература по разработке программного обеспечения ясно указывает на ключевую роль инженерии требований в жизненном цикле разработки программного обеспечения. В некоторых случаях для приоритизации требований использовался подход нечеткой логики. Дефекты программного обеспечения также могут быть обнаружены с помощью нечеткой логики. Нечеткая логика также используется для оценки стоимости, рисков, надежности и общих трудозатрат, необходимых для разработки. Нечеткая логика также использовалась для разработки интеллектуальной системы рекомендаций, с помощью которой находить требования в неформальных определениях крупномасштабных программных проектов.

Основная цель спецификации требований — не допустить упущения каких-либо обязательных требований. Чакрабурти и др. [1] в своем исследовании подчеркнули, что разработка требований является важнейшим этапом жизненного цикла разработки программного обеспечения. Они также отметили, что этот этап используется для преобразования недостающих потребностей и запросов потенциальных пользователей программного обеспечения в полные, точные и формализованные функции.

Бергин и др. [2] в своем исследовании попытались показать, как возникает неопределенность в разработке программного обеспечения и как эта неопределенность может отражаться при измерении качества программного обеспечения. Они также подчеркнули, что измерение программного обеспечения играет решающую роль на всех этапах жизненного цикла программного обеспечения.

Лима и др. [3] исследовали один из важнейших вопросов, связанных с эффективностью разработки программного обеспечения, — приоритетность выполнения требований. В этой работе было обнаружено, что приоритизация требований имеет неоднозначные аспекты, поэтому для более точного представления и решения этой проблемы были предложены концепции нечеткой логики.

Ядав и др. [4] предложили модель, которая предсказывала количество дефектов конструкции до этапа тестирования. В этой работе

Метрики прог рамного обес печения учитываются при разработке моделей раннего прог нозирования ошибок. Метрики размера прог рамного обес печения и результаты анализатребованийиспользуются для прог нозирования количества возможных дефектов во время тестирования с использованием правил нечёткой логики. В данномис следованиииспользуются 20 наборов данных реальных прог рамных проектов для демонстрациии обоснованности и практичности предлагаемого подхода.

В работе Хуанга и соавторов [5] была предложена новая нейронечёткая модель оц енки стоимости прог рамного обес печения. Эта модель обладает некоторыми характеристиками нейро-нечёткого подхода, такими как обучаемость их орошая интерпретируемость.

Нисаридр. [6] сосредоточились на оц енке трудозатрат, направленной на определение количества рабочих часов и сотрудников, необходимых для разработки проекта. Цельюих исследования был анализиспользования нечёткой логики в существующих моделях оц енкистоимости, а также детальное изучение прог рамного обес печения и методов оц енки проектов, доступных в отрасли, с оц енкой их сильных и слабых сторон.

Нечёткая модель, разработанная Альджадали и соавторами [7], была разработана для оц енки надёжности прог рамных проектов. В их собственных нечётких моделяхиспользовался метод Такаги-И-Сугено. В данномис следовании нечёткие модели были протестированы в трёх типах приложений: системах управления в реальном времени, военных системах приложения и операционные системы.

Альзуби и др. [8] предложили новую тему рекомендаций, основанную на алгоритме Априори, для выработки требований пользователей. В данномис следованиииспользовался набор данных, содержащий 4000 записей. Созданные пользователи миправила активируются для анализа неформальных текстовых данных с целью выработки требований для пользователей.

Бубенко и др. [9] в своей работе обобщили расширенную парадигму моделирования требований и информации, основанную на их взаимосвязанной метамодели. В данномис следовании инженерия требований была определена как систематический процесс разработки требований для анализа проблемы, документирования полученных результатов и проверки их точности. Эта амбициозная работа была сосредоточена на процессе повторного использования и поддерживала необходимость наличия инженера по повторному использованию ответственного за проектирование в процессе разработки.

Егоров и др. [10] исследовали использование функций Т-нормы, определяющих нечёткое пересечение множеств и конъюнктивных нечёткой логике. Они также попытались продемонстрировать, как эти функции могут быть использованы в спецификации требований. Особое внимание было уделено проверке требований в ходе приёмочных испытаний информационных систем. Они пояснили, что целью проверки является определение качества прог рамного продукта путём проверки его соответствия функциональным и нефункциональным требованиям.

Рамзан и др. [11] отмечают некоторые серьёзные недостатки существующих методов приоритизации требований. В данномис следовании предлагается интеллектуальный метод, основанный на нечёткой логике, для приоритизации требований. Этот методиспользует нечёткую логику для приоритизации требований на основе их воспринимаемой ценности. Рекомендуется, чтобы система создавала два отдельных документа с требованиями в порядке приоритета.

Первый документ содержит требования, которым присвоен более высокий приоритет, чем определённый порог, а второй с низким содержанием требований, которые имеют более низкий приоритет, чем определённый порог. заданный порог. Их подход был базовым, и они подтверждали, что логический метод приоритизации требований получит гораздо более широкое распространение, если будет прост в использовании и понятен.

В работе Эбраерта и др. [12] была предпринята попытка объединить этапы проектирования и внедрения с помощью основанного на изменениях

Диаграммы FODA для проектирования линейки продуктов. Проектирование линейки прог рамных продуктов — это парадигма разработки прог рамного обес печения, которая поощряет повторное использование на всех этапах разработки прог рамного обес печения. Роль диаграммы FODA заключается в кратком описании допустимых комбинаций функций в системе. Диаграммы FODA расматриваются как механизм, позволяющий заполнить пробел между требованиями и проектом. При таком подходе диаграмма FODA оказалась полезной не только для соединения

Требования и проектирование, а также связь между проектированием и реализацией. В своёмис следовании они предложили метод, который можно использовать для автоматического создания диаграммы FODA на основе изменений в исходном коде. Бенавидес и др. [13] также работали над проектированием линейки продуктов. Они подчеркнули, что проектирование линейки продуктов прог рамного обес печения доказало свою эффективность в производстве прог рамного обес печения. Они выступили за моделирование функций, направленное на выявление сходств и различий между всеми продуктами линейки прог рамного обес печения. Модели функцийиспользуются для моделирования линейки прог рамного обес печения с точки зрения функций и взаимовыигрыша между ними.

Гонсалвес и др. [14] сосредоточились на семантической стороне нечёткости. Согласно их работе, прилагательное «лёгкий» является неопределённым, а значит, нечётким термином, поскольку удобствозависит от предпочтений пользователя. Лёгкий курс можно определить как курс, где все студенты получают высокие оц енки. Высокая (высокая оц енка) — неопределённый термин, что также делает его нечётким. Авторы подчеркнули, что их целью было создание автоматизированных инструментов разработки прог рамного обес печения для разработки приложений с нечёткими требованиями. В ихис следовании предлагается метод разработки приложений, поддерживающих нечёткие требования.

Подчеркнув важность анализа требований, Лю [15] описывает способ анализа функциональных требований с точки зрения их одних данных, вых одних данных и их взаимосвязей. Авторы жаловались на то, что во многих проектах по разработке прог рамного обес печения требования иногда не были детально прописаны, что значительно затрудняет верификацию и поддержку прог рамного обес печения. Основная сложность заключалась в том, что многие требования к продукту изначально были нечёткими.

Заказчики часто определяли требования в нечетких терминах, таких как «хорошо», «высокий», «низкий» или «очень важно».

Хисие и др. [16] утверждают, что разработка прог рамного обес печения изначально связана с сложностью неопределённости и риском. Анализ рисков — важнейший этап разработки прог рамного обес печения, однако оц енке рисков часто уделяется недостаточное внимание. Руководителям необходимы более эффективные инструменты для снижения высокого уровня отказов в прог рамных проектах. В этом случае для анализахорошо подходит нечёткая логика. Этапы оц енки рисков:

- Информациисобирается.
- Риски определены.
- Определены функции принадлежности (для нечеткой логики).
- Рискоцениваются по шкале (экстремально высокий, очень высокий, высокий, довольно высокий, средний, довольно низкий, низкий)
- Вес оцениваются. (Нечеткая логика)
- Оценкариса завершена.

Этот метод оц енки риска, основанный на нечеткой логике, утверждает, более высокая точность, сокращение времени расчетов и уменьшение количества ошибок.

III. ГДЕ ПРИМЕНЯТЬ НЕЧЕТКУЮЛОГИКУ

А. Системы нечеткой логики
Нечёткая логика — это общая вычислительная система, вдохновлённая человеческим мышлением. Эта система основана на отношениях между лингвистическими переменными и логическими выражениями. Нечёткая логика считается искусственным

Подраздел интеллекта. Нечёткая логика основана на теории нечётких множеств. Нечёткие множества — это множества с размытыми границами. Проверка принадлежности к нечёткому множеству может возвращать любое значение от «истина» до «ложь».

В то время как в классической логике событие можно обозначить 0 или 1, в нечёткой логике можно обозначить бесконечное множество значений между 0 и 1. В нечёткой логике утверждение обладает определённой степенью истинности или ложности. Утверждение может быть истинным или ложным в определённой степени, то есть оно может быть истинным, ложным, слегка истинным или слегка ложным.

Нечёткая логика в настоящее время используется во многих сферах повседневной жизни. Сокращает время ожидания, оптимизирует пассажиропоток при ожидании лифта. Определяет оптимальную точку остановки и освещение, когда на камерах одновременно находится несколько объектов. Визор. В стиральных машинах системы нечёткого управления определяют степень загрязнения и вес белья, а также тип ткани и автоматически выбирают наиболее подходящую программу стирки. В пылесосах нечёткая логика отслеживает состояние поверхности и окружающей среды и с соответствующим образом регулирует мощность двигателя. В водонагревателях она регулирует нагрев в зависимости от количества и температуры используемой воды. В кондиционерах она определяет оптимальные рабочие условия, оптимизирует условия окружающей среды, и увеличивает мощность охлаждения, когда кто-то входит в помещение.

Программное обеспечение Fuzzy регулирует контрастность, яркость и цвет экранов умных телевизоров, оптимизирует условия освещения.

Простая конструкция нечёткой системы показана на рис. 1. На этапе фазификации действительные числовые значения преобразуются в значения принадлежности к нечётким множествам. Затем, с помощью механизма нечёткого вывода, правила типа ЕСЛИ-ТО преобразуются в нечёткое отношение, определяемое между входом и выходом простыми трансформациями. На этапе дефазификации нечёткое множество преобразуется обратно в значения реального мира. Таким образом, обеспечивается чёткое соответствие принадлежности.

Дефазификация значений обычно вычисляется путём взятия средневзвешенных значений.

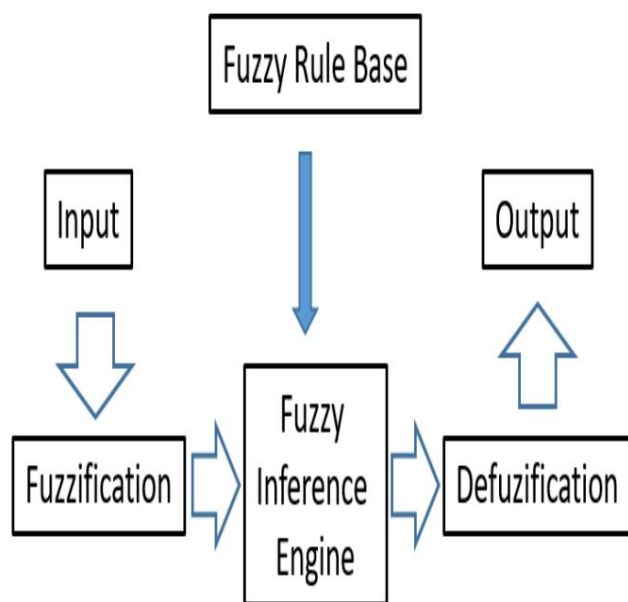


Рис. 1: Конструкция базовой нечёткой системы.

Б. Зачем использовать нечёткую логику

Анализ требований — это первый этап разработки программного обеспечения. Процесс разработки критически важен. Самый большой риск в программном проекте — разработка некорректного продукта. Управление требованиями должно быть точным и эффективным.

Физические условия, законы, условия конкуренции и часто пожелания заказчиков играют важную роль в определении требований. Нежелательные и дублирующие требования могут помешать определению объема и стоимости проекта. Требования должны быть сформулированы максимально ясно и на естественном языке. Неверный анализ требований может привести к принятию неверных решений и даже к остановке проекта. Неверные решения, принятые на этапе определения требований или на ранних этапах проектирования, обходятся дороже, чем ожидается, будут исправлены позже. Чтобы предотвратить подобные нежелательные...

В некоторых ситуациях анализ требований должен проводиться точно и эффективно, а неохотимые решения должны приниматься как можно раньше и с минимальными затратами.

Чтобы определить точные области применения ИИ в инженерии требований, необходимо представить себе интерактивную среду разработки, поддерживаемую системой поддержки принятия решений, имеющей доступ к информационной базе проекта. Разработчик может запросить определённые типы информации из такой базы.

В этом контексте мы можем предположить, что разработчик либо пытается определить некоторые аспекты текущего состояния проекта, либо пытается решить, каким задачам следует отдать приоритет для наиболее эффективного процесса разработки.

Следует также отметить, что планирование разработки программного обеспечения — очень динамичная задача, поскольку быстрое прототипирование, применяемое в сочетании с небольшими циклами валидации и верификации, является очень популярным методом (так называемым эволюционным процессом разработки программного обеспечения), направленным на снижение рисков при разработке больших и сложных программ. В такой гибкой модели процесс некоторые задачи, естественно, могут находиться в полузавершённом состоянии, а модули — в полупункциональном. Оба эти фактора потенциально нечёткие и могут привести к другим факторам нечёткости, таким как относительная стоимость или риск. Однако вычисление нечётких значений истинности для программного обеспечения — непростая задача, поскольку программное обеспечение имеет некоторые особенности по сравнению с физическими артефактами.

Использование нечёткой логики в проектировании требований также требует принятия стратегии для работы с некоторыми присущими неструктурированности требований к программному обеспечению потенциальными возникающими из разных источников. Во-первых, программное обеспечение — это абстрактный артефакт, который может быть структурирован в различных формах. Например, в объектно-ориентированном проектировании может быть несколько способов разделения проекта на классы и интерфейсы. У заинтересованных сторон могут быть совершенно разные представления об общей архитектуре проекта и его компонентах. Это может создать ощущение несовместимости между требованиями и даже противоречия. Между модулями могут существовать отношения включения, перекрытия или зависимости, что делает традиционные нечёткие модели неполными. Распространённой ошибкой является предположение, что разработчики просто разрабатывают требования по одному за раз. К сожалению, задачи кодирования (или проектирования) и

Требования — это совершенно разные концепции, разделённые уровнем абстракции. Задача может удовлетворять нескольким требованиям, в то время как требование может быть разделено по нескольким задачам. Иногда требование, такое как стандартизация или соглашение об именовании, может быть разделено по всему проекту, как и в некоторых сквозных аспектах. Задачи также могут возникать динамически и принимать форму модификации, а не разработки с нуля.

Некоторые требования, такие как требования к производительности, очень легко выразить в виде нечетких логических утверждений. Поскольку они поддаются количественной оценке. Единственная необходимая задача — сравнить числовые данные с предопределёнными нечеткими множествами. Некоторые другие требования, записанные обычным текстом, используют нечеткие термины, которые можно использовать при создании формальных нечетких требований. Нечеткие термины, такие как «большой», «маленький», «короткий», «длинный», «важный», «очень важный», используются для выражения экспертных знаний уже в форме нечетких правил. Их следует проверять только на внутреннюю согласованность.

В качестве примера можно представить определение требования, например, «Для ввода в систему выполняется проверка авторизации». Рассмотрим, как такое требование можно интегрировать в нечеткий контекст. Хорошей идеей будет добавить несколько нечетких терминов для размытия спецификации требования, например: «Очень важно иметь авторизацию для ввода в систему». В такой формулировке спецификация требования не только утверждает, что авторизация является подмодулем ввода в систему, но и содержит метаутверждение о её важности. Фактически, это также косвенное указание на то, что система могла бы обойтись без проверки авторизации, но (хотя и возможно) это было бы крайне неприятной альтернативой, получающей низкие значения валидности.

IV. НЕЧЕТКАЯ БАЗА ЗНАНИЙ ДЛЯ УПРАВЛЕНИЯ ТРЕБОВАНИЯМИ

Динамически управляемый набор требований требует наличия менеджера требований, взаимодействующего с интегрированной средой разработки на протяжении циклов развития проекта. На рис. 2 представлена контекстная модель нашего менеджера требований. Рисунок представляет собой UML-диаграмму с несколькими добавленными стрелками потоков данных. Из этой архитектуры очевидно, что менеджер требований играет центральную роль в разработке.

Каждый проект хранится в Configuration Manager вместе с набором требований и набором тестов. По мере проведения тестов генерируются данные обратной связи, позволяющие отслеживать выполнение требований.

В ходе вычислений также используется база онтологий для предоставления контекстной информации об отношениях между некоторыми данными нечетких операций.

Онтологические знания, как и любая повторно используемая сущность, хранятся в репозитории повторного использования и регулярно обновляются менеджером по требованиям.

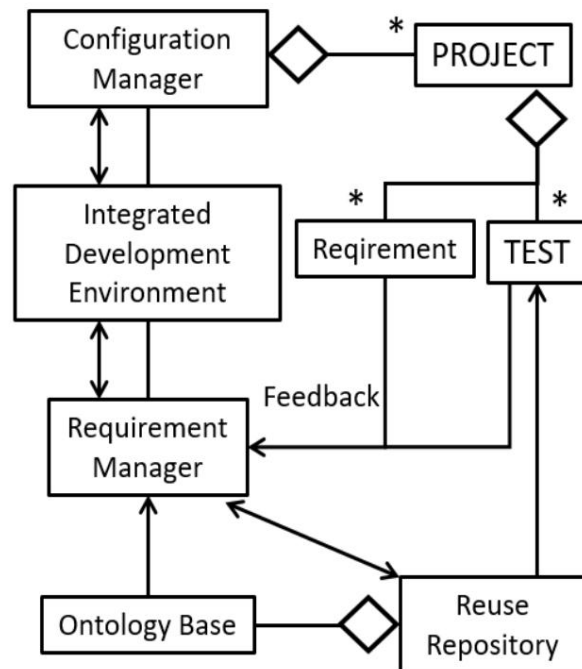


Рис. 2: Контекстная модель менеджера требований

В центре всей архитектуры находится интегрированная среда разработки, которая управляет всем процессом.

Здесь код разрабатывается и тестируется во взаимодействии с программистом. Информация о текущем состоянии задач предоставляется постоянно.

Наш подход к нечеткой логике основан на нашем оригинальном унифицированном нечетком операторе конъюнкции [17] и нечеткой версии значений истинности Бенлапа (логические константы логики Бенлапа). [18]. Вместе они образуют довольно сложную пару, подробности работы которой выходят за рамки этого документа.

Для представления требований мы структурируем нечеткие операции вокруг нечеткого обобщения диаграмм FODA [19]. FODA означает Feature Oriented Domain Analysis (анализ предметно-ориентированных областей), и диаграммы FODA очень полезны для определения логической изменчивости системы, представляя композицию признаков в виде иерархии, где точками соединения являются логические операторы. Эта нотация также используется для выражения обязательных и необязательных узлов.

Серьезная попытка полуавтоматизированного управления требованиями в реальном проекте не только создает нечеткие значения, но и вынуждена работать с неполными или противоречивыми данными. Значения истинности в такой среде должны быть более размытыми, чем стандартные нечеткие значения.

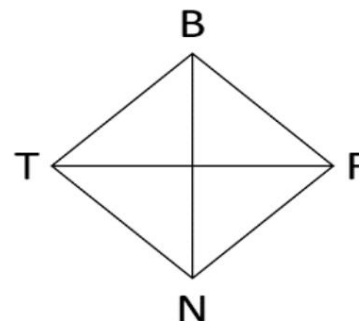


Рис. 3: Значения истинности Бенлапа [18]

На рис. 3 показаны четыре логические константы в логике Бенлара. Два значения слева и справа — «Истинно» (T) и «Ложно» (F) соответственно. В означает, что оба варианта — «Истинно» и «Ложно». Удерживать, пока N означает, что ни одно из них не может быть признано истинным. Таким образом, значения мня вля ются «истина», «ложь», «оба» и «ни то, ни другое».

В этом отношении «True» означает, что True выполнено, а False не выполнено. Аналогично, «False» означает, что False выполнено, а True не выполнено. Это омомента вс ёпроис х одит так же, как в клас ичес кой логике. Однако значение В утверждает, что и True, и False выполня ются одновременно. В то время как значение N утверждает, что ни True, ни False не выполня ются.

На самом деле, нечёткое обобщение этих значений часто бывает проще для понимания. В этом случае не только значения в углах ромба, но и точки повс ей его поверх ности рас матрива ются как допустимые значения логической переменной.

Линия между T и F представля ет с собой клас ичес кую размытость. Область под этой линией с соответс твует области, где переменная не достаточно ограничена (имеет некоторую свободу изменения с воего значения). Такое представление вносит в значение измерение интуиционизма, и значение состоит из трёх действительных чисел, обозначающих степень истинности, ложности или неизвестности. С другой стороны, область над линией представля ет случай, когда рас матриваемая переменная чрезмерно ограничена. В этом случае измерение параконсистентности (истина и ложь) (с осущ ествующий) интегрирован с означением, а третье значение, наряду с True и False, представля ет с собой степень противоречия. В результате значение истинности двумерно: одно указывает на положение на шкале True-False, а другая — на ос и Противоречие-Неизвестное.

Двумерные нечёткие значения истинности позволяют нам создавать более сложные интенсификации для дерева признаков. Запросы могут быть условно безопасными или рискованными. Обычный запрос, отправляемый в обычную базу знаний, будет иметь единственный интенсификации, который может быть представлен дискретным значением «Истинно». Запрос должен возвращать значение «Истинно»; это единственный критерий успешной работы дискретной базы знаний. Двумерные нечёткие значения истинности, с другой стороны, могут претендовать на точность, согласованность, оптимизм или пессимизм при необходимости. Например, неизвестный компонент нечёткой переменной может быть оценен как «Истинно» или «Ложно» в зависимости от текущей степени оптимизма/пессимизма в вычислениях.

Этот параметр настройки можно изменять в локальном правиле или он может быть глобальной переменной среды, хранящейся в пакете.

Для иллюстрации нашего случая представим аналогичную ситуацию с системой компьютерной диагностики, использующей нечёткую логику. Такие системы используются для таких серьёзных задач, как диагностика рака, поэтому худший результат, которого следует избегать любой ценой, — это ложноположительный результат, означающий, что экспертная система диагностировала пациента как здорового, хотя у него был рак. Мы не хотим этой фатальной ошибки, поэтому хотим, чтобы система склонялась к значению «истина» (ракоположительный) в случае любых сомнений. (Технически это называется оптимизмом, поскольку склоняется к значению «Истина», хотя семантически это пессимистическая точка зрения). Также можно делать систему, например, 0,7 оптимистичной или склонить её по диагонали, например, к значениям «Ложь» и «Неизвестно» одновременно. Такое обобщение нечёткой логики полезно для моделирования гуманистических приоритетов при вычислениях в реальной среде.

Ещё одно дополнение, которое мы сделали к классическому нечёткому

Логика заключается в реализации нечётких логических операторов. В нечёткой логике нечёткие функции и-нормы и т-нормы используются для вычисления одних данных логических операторов (И и ИЛИ). (с соответствием). Эти функции называются истинностно-функциональными. Это означает, что им не требуется никаких одних данных, кроме двух значений истинности. Однако, как мы обсуждали в [16], реалистичное вычисление нечётких значений в реальной жизни требует контекстного знания точного семантического отношения между двумя (или более) операторами.

По этой причине мы считаем, что простое дерево FODA недостаточно для представления вычислений, связанных с выполнением требований, даже несмотря на то, что листья дерева содержат нечёткие значения. Дерево необходимо дополнить, добавив ассоциаций, указывающие на семантическое состояние между ветвями (горизонтальные связи).

Как показано на UML-диаграмме (рис. 4), задачи менеджера требований выводятся из расширенного дерева. Задача — это значение, отсуживаемое системой в процессе разработки. Это значение вычисляется на основе ряда других параметров с соответствием с рекомендациями, определяемыми как расширенные деревья. В системе есть три встроенных задачи: завершенность, функциональность и качество.

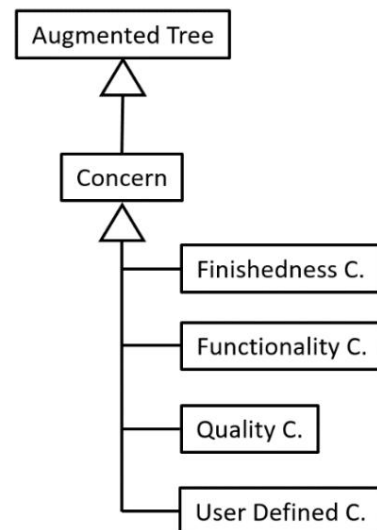


Рис. 4: Проблемы в менеджере требований

Работа о завершении показывает с соотношением общих эффективных усилий, затраченных на проект, к общим усилиям, необходимым для его завершения.

отражает степень выполнения программным обеспечением своих функций. Quality Concern оценивает общее качество артефакта, используя предопределённые показатели качества, включая качество кода.

Четвёртый тип задач — это пользовательские задачи. Пользователь может добавлять любое их количество. Некоторые из них могут включать параметры всего проекта, другие — локальные параметры. Разработчик может добавлять задачи для определённых типов функциональности, таких как «взаимодействие с пользователем» или «подключение».

Также можно отсуживать функциональность от отдельных подсистем. Например, разработчику может быть интересно узнать, насколько функциональна подсистема регистратора данных.

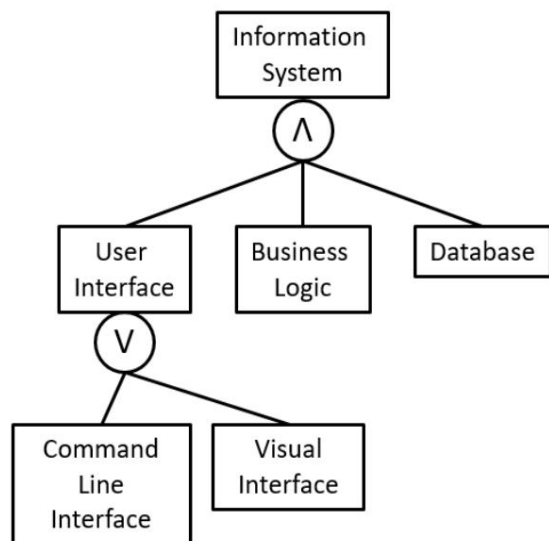


Рис. 5: Вариант синтаксиса FODA

Наш синтаксис для представления проблем основан на FODA. Диаграммы. Чтобы семантические расширения не выглядели слишком сложными, в синтаксис внесены некоторые изменения, делающие его более расширяемым. Как видно на рис. 5, логические операторы обозначены маленькими кружками, соединяющими дочерние элементы с соответствующим узлом. Роли конъюнкции и дизъюнкции те же, но теперь можно легко добавить агрегацию и другие операторы.

Наибольшее отличие наших вычислений на основе нечёткой логики от классических подходов нечёткой логики заключается в использовании операторов, не являющихся истинностно-функциональными, подробности которых можно найти в [16]. Подводя итог идее этого подхода, можно сказать, что корреляция между двумя утверждениями учитывается при вычислении логического оператора, объединяющего эти два утверждения. Таким образом, в этом подходе операнды — это не просто числа, а информация о самих утверждениях, которая должна быть сохранена. Именно поэтому для корректного вычисления требований требуется база онтологий.

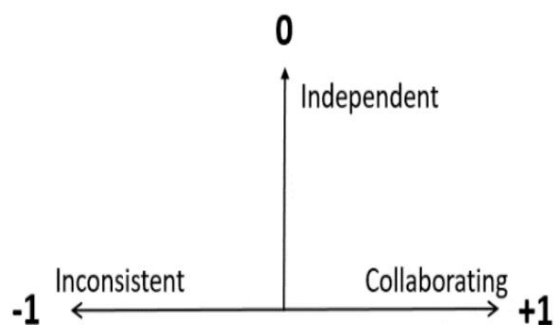


Рис. 6: Направления взаимодействия утверждений

Рис. 6. показывает возможную вариацию корреляции. между любыми двумя конечными. Конечные могут быть взаимодействующими в том смысле, что высшее значение одной конечной может быть признаком высшего значения другой (корреляция = 1). Они могут быть несовместимыми, так что высшее значение в первой конечной препятствует высокому значению другой (корреляция = -1).

Они также могут быть независимыми друг от друга (корреляция = 0). Как указано в нашей предыдущей работе [16], семантически Т-норма Гёделя является подходящей функцией для вычисления конъюнкции в случае взаимодействия операндов (это также наиболее часто используемая Т-норма функция min). В случае независимых операндов правильным выбором является Т-норма произведения. Если операнды противоречивы, следует предпочесть Т-норму Лукасевича.

Для случаев, лежащих между этими двумя, эти формулы следует объединить (по крайней мере, две формулы по отдельности с Т-нормой произведения). Мы использовали параметрическое семейство общих Т-норм, называемое Т-нормами Франка для плавного комбинирования, а также немного изменил формулу, чтобы разрешить наше значение корреляции (между -1 и 1) как подходящий параметр в норму Франка. Формула. Это обеспечивает плавное изменение значений между дискретными значениями корреляции. Все Т-нормы также имеют соответствующие Т-конормы, которые можно вычислить непосредственно из формулы Т-нормы. Также можно вычислить соответствующую материальную импликацию.

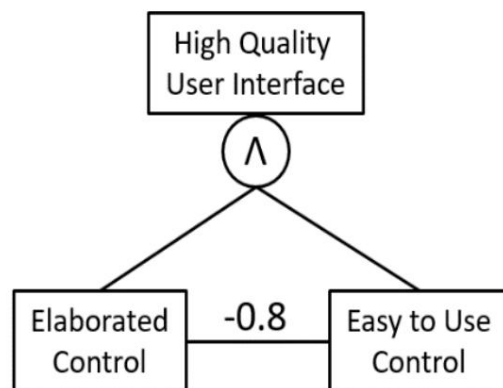


Рис. 7: Концептуальное соединение

На рис. 7 представлен фрагмент дополненного дерева, указывающий на нечёткую конъюнкцию. Конъюнкция определяется с помощью «Высококачественный пользовательский интерфейс» путём объединения «Продуктового элемента управления» с «Простым в использовании элементом управления». Линия между операндами указывает на корреляцию между этими двумя свойствами, а значение -0,8 означает, что они практически противостоят друг другу (значение близко к -1). В этом случае менеджер по требованиям будет использовать формулу, очень близкую к Т-норме Лукасевича, с небольшим добавлением Т-нормы продукта.

Подобные компромиссы в дизайне — естественный источник отрицательной корреляции, но на самом деле именно в них и рождается суть хорошего дизайна. Когда две силы противодействуют друг другу и создаётся напряжение, возникает творчество, причём не только в природе, но и в человеке.

Другим источником отрицательной корреляции, конечно же, является множественность источников требований. В принципе, в проекте может быть слишком много заинтересованных сторон с конфликтующими интересами. Например, в гипотетической школьной информационной системе студенты, преподаватели, менеджеры, кафедра, факультет и студенческое бюро — все хотят иметь возможность контролировать и управлять всем, что может привести к конфликтам. Преподавание студентам может включать повышение безопасности доступа к данным, в то время как преподаватели могут требовать большей скорости.

Свобода действий при этом. Без этих проблем нам не нужны были бы дизайнеры и инженеры.

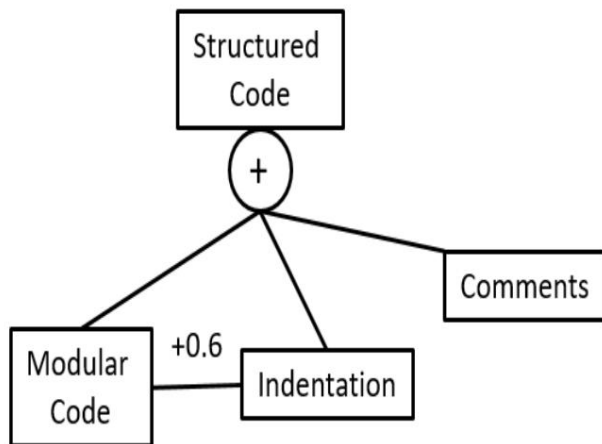


Рис. 8: Агрегация с перекрытием

Положительная корреляция также может мешать правильному вычислению проблем. На рис. 8 представлен еще один фрагмент расширенного дерева, демонстрирующий определение проблемы, называемой «структурированный код». Вместо конъюнкции или дизъюнкции эта функция определена с помощью оператора агрегации. Агрегация применяется в тех случаях, когда несколько взаимодействующих друг с другом участников выполняют разные части одной и той же задачи. Хорошим примером служит то, как пиксели изображения влияют на общий цвет изображения. Обычный способ реализации агрегации — оператор усреднения (обычно взвешенного усреднения). Когда операторы агрегации семантически перекрываются, их вес следует соответственно уменьшить.

В этом примере структурированный код был определен как комбинация модульного кода, отступов и комментариев. Между модульным кодом и отступами наблюдается положительная корреляция. Это связано с тем, что при модулизации кода фрагменты скопированного кода заменяются вызовами функций, блоки становятся меньше, а проблема отступов также частично решается. В этом примере предполагается, что модульный код и отступы перекрываются на 0,6, и их веса будут вычисляться соответствующим образом. Для комментариев корреляция не указана, поэтому предполагается нулевая корреляция.

Возможны случаи, когда два требования, заданные двумя разными заинтересованными сторонами, полностью перекрываются (корреляция = 1). Одно требование может полностью одержать другое, и в этом случае меньшее из них следует отбросить. Если онтология не содержит информации об отношении включения, два значения можно усреднить по отдельности и затем включить в более крупное агрегирование как единое целое. Если одно требование представляется собой простое переопределение одного и того же явления, проблему следует устранить, удалив одно из них.

Оператор агрегации можно модифицировать и другими способами, как показано на рис. 9.

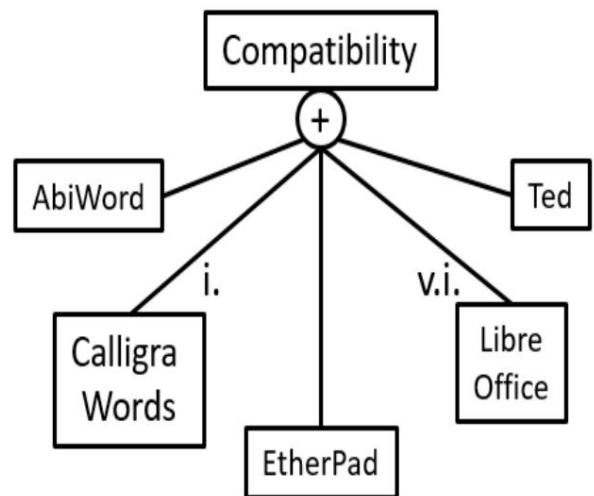


Рис. 9: Совместимость как совокупность характеристик

На рис. 9 представлено требование к совместимости данных для текстового процесса. Разрабатываемый новый текстовый процессор должен быть совместим с различными существующими свободно доступными текстовыми процессорами. Дерево задач вычисляет общую степень совместимости нового продукта с точки зрения его степени совместимости с каждым из существующих текстовых процессов. Все эти значения нечеткие, поскольку наборы поддерживаемых файлов могут пересекаться в различной степени.

Ни одна из функций дерева не является по-настоящему незаменимой, но мы не можем претендовать на дизъюнктивную комбинацию.

Дизъюнкция показывает, что даже одно одностороннее отношение в данном случае ситуация скорее напоминает «чем больше, тем лучше».

Однако дизайнер может посчитать, что один или два из этих участников немного важнее. Это можно отразить в дереве, добавив метки «i» (важно).

и «vi» (очень важно) на значениях операндов. В этом случае менеджер требований все равно вычислит значение агрегации, но внесет необходимые нечеткие коррективы в весовые коэффициенты соответствующих операндов, насколько это позволяло нечеткие термины «важно» и «очень важно».

V. ЗАКЛЮЧЕНИЕ

Разработка требований — чрезвычайно важная часть разработки программного обеспечения, и в крупных проектах автоматизированная поддержка просто необходима. Это также динамичный процесс, включающий масштабируемое поведение, основанное на масштабируемых суждениях. Разработчикам также приходится иметь дело с взаимосвязанной, избыточной, неполной или противоречивой информацией.

В качестве решения этой проблемы в данной работе представлена логическая структура нового инструмента управления требованиями на основе нечеткой логики с упоминанием его наиболее впечатляющих решений. Это оригинальное двумерное представление нечетких значений и наш новый семантически-ориентированный метод выполнения нечетких операций, не основанный на истинностно-функциональном подходе. Они были объединены в расширенную версию диаграмм FODA, которая позволяет отражать дополнительные семантические характеристики.

В рамках будущей работы мы надеемся завершить реализацию нашего инструмента и наблюдать его разумное поведение.

ССЫЛКИ

ОБ АВТОРАХ

- [1] А. Чакраборти, М. Баовали и А. Арефин, «Роль проектирования требований в жизненном цикле разработки программного обеспечения», Журнал новых тенденций в вычислительной технике и информационных науках, 3(5), 2012.
- [2] М. Бергин и Дж. Дебнат, «Размытость и неточность в программной инженерии», Всемирный конгресс по автоматизации, 24-26 июля 2006 г.
- [3] Д. Лима, Ф. Фрейтас и Г. Кампос, «Нечеткий подход к приоритизации требований», Springer, 2011.
- [4] Д. Ядав, С. Чагурведи и Р. Мисра, «Прогнозирование ранних дефектов программного обеспечения с использованием нечеткой логики», Международный журнал по инженерингу и производительности 8(4), 2012, стр. 399-408.
- [5] Х. Хуан, Л. Капрец и Дж. Рен, «Нейронно-нечеткая модель для оценки стоимости программного обеспечения», Труды Третьей международной конференции по качественному программному обеспечению IEEE, 2003.
- [6] М. Нисар, Й. Ван и М. Элах, «Оценки усилий по разработке программного обеспечения с использованием нечеткой логики – обзор», Пятая международная конференция по нечетким системам и обнаружению знаний, IEEE, 2008, стр. 421-427.
- [7] С. Альджадиди, А. Шета, «Прогнозирование надежности программных систем с использованием нечеткой логики», Восьмая международная конференция по информационным технологиям: Новые поколения, 2011.
- [8] С. Альзуби, Б. Хавашин и М. ЭльБес, «Новая система рекомендаций, основанная на априорном алгоритме для разработки требований», Пятая международная конференция по анализу, управлению и безопасности социальных сетей (SNAMS), 2018 г., стр. 323-327.
- [9] Дж. Бубенко, К. Ролланд и П. Лукопулос, «Содержание моделирования нечетких и формальных требований», IEEE, 1994.
- [10] Ю.С. Егоров, В.Р. Милов, А.С. Квасов, «Формализация требований к программному обеспечению информационных систем с использованием нечеткой логики», Международная конференция «Информационные технологии в бизнесе и промышленности 2018», Издательство IOP, 2018, стр. 1-5.
- [11] М. Рамзан, М. Арфан Джаффар М. Амджад Икбал, «Приоритизация нечетких требований на основе ценностей и ее структура оценки», Четвертая международная конференция по инновационным вычислениям, информации и управлению 2009 г., стр. 1464-1468.
- [12] П. Эбраерт, Д. Сеттенс и Д. Янсенс, «Схемы FODA на основе изменений, сокращающие разрыв между проектированием, ориентированным на функции, и их реализацией», Конференция: Труды симпозиума ACM 2011 года по прикладным вычислениям (SAC), Тайчжун, Тайвань, 21-24 марта 2011 г.
- [13] М. Гонсалвес, Р. Родригес, Л. Тинео, «Формальный метод реализации нечетких требований», Дуна (Медельин, Колумбия), 173(III), 2012, стр. 15-24.
- [14] Х. Лю «Нечеткие требования», IEEE Potentials, Институт инженеров электротехники и электроники (IEEE), апрель 1998 г.
- [15] M. Hsieh, Y. Hsu и C. Lin, «Оценки рисков в новых проектах разработки программного обеспечения на фронте: подход нечеткой логики», Журнал окружающего интеллекта и гуманизированных вычислений, 9(2), Springer, апрель 2018 г.
- [16] А. Эгесой, «Выбор нечетких операторов для реальных инженерных приложений», Турецкий журнал нечетких систем, 8(2), 2017, стр. 73-89.
- [17] ND Belnap, «Плезная четырехзначная логика», в: G. Epstein и JM Dunn (ред.), Modern Uses of Multiple-Valued Logic, Reidel, Dordrecht, 1977, стр. 7-37.
- [18] П. Похьялайнен, «Выражения анализа предметной области, ориентированные на признаки», Компьютерные науки, 2008.



Ахмет Эгесой (доктор философии в области компьютерной инженерии) — преподаватель и доцент кафедры компьютерной инженерии ЭГЭ.

Университет Измира, Турция.

Научные интересы включают объектно-ориентированное проектирование, шаблоны проектирования программного

обеспечения на основе моделей, разработку, искусственный интеллект, языки программирования, парадигмы программирования, философия языка, семиотику и представление знаний.



Айлин Гюкчел (магистр наук в области компьютерной инженерии) — аспирант кафедры компьютерной инженерии Эгеской университета, Измир, Турция. Научные интересы включают программную инженерию объектно-ориентированное проектирование, нечеткую логику и шаблоны проектирования.